Joint Optimization of Service Deployment and Request Routing for Microservices in Mobile Edge Computing

Kai Peng, Liangyuan Wang, Jintao He, Chao Cai, and Menglan Hu

***Abstract*—Microservices as an emerging architecture are creating new opportunities to enable superior network services in Mobile Edge Computing (MEC). In the presence of huge amounts of user requests, the massive communications among microservices have become notoriously complicated. Due to the intricate data dependencies of the microservices, the overall performance of large-scale MEC applications simultaneously depends on both service deployment and request routing. However, most existing work ignores the interdependencies of microservices and studies the deployment and routing as two isolated problems. In this case, this article investigates the joint optimization of service deployment and request routing in edge computing. We first formulate a delay minimization problem via mixed integer linear programming and queuing analysis, and then provide a hardness proof on the problem. In addition, this article presents a 2-approximation algorithm, followed with rigorous mathematical proofs to demonstrate the approximation ratio. The proposed two-phase algorithm consists of rounding based service deployment and adaptive-scaling-based request routing policies, which employ fine grained joint optimization to minimize service response delay. Finally, we illustrate the near-optimal performance of the proposed algorithm via comprehensive experiments.**

***Index Terms*—Services computing, approximation, service deployment, request routing, mobile computing.**

I. Introduction

MICROSERVICES as a new Web services model has attracted extensive research attention. The most distinctive feature is to decouple a monolithic application into multiple independent fine-grained services, which operates independently while coordinating with each other [1]. Due to its strong reliability and scalability, microservice has been widely developed and deployed in Mobile Edge Computing (MEC) scenarios [2]. For instance, leading companies including Spring and Alibaba have developed microservice frameworks [3], [4] to meet the increasing needs for MEC applications.

In MEC network applications, unlike traditional Virtual Network Functions (VNFs), microservices are decoupled and lightweight. Therefore, user requests are typically handled by multiple microservices together [5]. These microservices are reused and engage in frequent communication while processing a substantial volume of user requests, thereby establishing a service call graph [6], [7]. In the context of a significant influx of user requests, the extensive intercommunication within these call graphs escalates in complexity. The overall performance of large-scale MEC applications depends on both service deployment and request routing, given the intricate data dependencies of the microservices. This involves optimizing microservice deployment location and quantity. As user requests are handled by microservice instances, they are probabilistically routed, allowing for single or multiple routing paths. On one hand, the effect of service deployment relies on the requests scheduled or routed among service instances in real-time. On the other hand, request routing requires deployed service instances as a precedent. Optimizing deployment alone reduces user request processing delay [8], [9], [10], while focusing solely on routing diminishes request propagation delay [11], [12], [13]. The localized optimization approach, however, could lead to an increase in service failure rates, particularly in delay-sensitive MEC applications. In this case, it is highly valuable to jointly optimize both microservice deployment and request routing for achieving superior service performance.

Nevertheless, in most existing work, service deployment and request routing are customarily studied as two isolated problems. Some prior work only investigates service deployment [14], [15], [16] while request routing is ignored. Other studies are concerned with request routing [17], [18], while service deployment is not mentioned. However, the overall performance of microservices depends on the interoperability of both deployment and routing, which requires joint optimization on both sides. Besides, a few previous papers simultaneously address both service deployment and request routing [19], [20], [21], but fine-grained deployment and routing are absent for the tightly-coupled twin problems. Although the above efforts have optimized the problem to some extent, the effect is not significant in terms of network service quality. Therefore, this

Manuscript received 18 March 2023; revised 28 November 2023; accepted 30 December 2023. Date of publication 3 January 2024; date of current version 12 June 2024. This work was supported in part by the National Key Research and Development Program under Grant 2022ZD0117104, in part by the National Natural Science Foundation of China under Grants 62171189 and 62272183, and in part by the Key Research and Development Program of Hubei Province, China, under Grants 2021BAA026, 2022BAA038, and 2023BAB074. *(Corresponding author: Menglan Hu.)*

Kai Peng, Liangyuan Wang, Jintao He, and Menglan Hu are with the Hubei Key Laboratory of Smart Internet Technology, School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: pkhust@hust.edu.cn; hust_wly@hust.edu.cn; hejintao@hust.edu.cn; humenglan@hust.edu.cn).

Chao Cai is with the College of Life Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: chriscai@hust.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3349408

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

paper emphasizes the critical interplay between microservice deployment and request routing, leveraging queuing networks for precise problem modeling. Additionally, we employ an approximate rounding strategy to achieve fine-grained deployment and routing.

To this end, this paper investigates the joint optimization problem of service deployment and request routing, while several practical challenges need to be addressed. First, the deployment and routing of microservices interact with each other, and the traditional divide-and-conquer approach is not applicable. Hence, the tight coupling between them must be taken into account when designing the algorithm. Second, due to the limited hardware devices and computing resources at edge nodes, they cannot be equipped with physical server clusters like cloud data centers. Therefore, how to reasonably choose the deployment location and number of microservice instances needs to be considered. Third, the service provider must have the capability to promptly process user-initiated requests. When user requests are initiated, they should be accurately routed to the appropriate microservice to prevent a large number of user requests from going unprocessed.

To address the aforementioned challenging issues, this paper presents a 2-approximation algorithm based on a rounding policy, followed with rigorous mathematical proofs to demonstrate the approximation ratio. The proposed two-phase algorithm consists of rounding based service deployment and adaptive-scaling-based request routing policies, which employ fine grained joint optimization to minimize service response delay. In summary, we make the following contributions:

- We present a set of formal models for microservice deployment and request routing. Their parameters are also defined and explained in detail. The model takes into account the practical characteristics of the network, such as computational resource capacity, service processing capability, and other constraints. Also, the processing delay and queuing delay of user requests in MEC networks are analyzed using queuing theory, and the Response Delay Minimization Problem (RDMP) is proposed.
- We propose a 2-approximation algorithm for service deployment and request routing. The algorithm first processes the optimal deployment solution through a rounding strategy to obtain the location and number of microservices. Then, the request routing is adjusted using the deployment probability with adaptive scaling. Finally, a feasible joint optimization scheme is obtained.
- We model the response delay minimization problem as an MILP and prove that it is NP-hard through reduction from the uncapped facility location problem. Furthermore, it is demonstrated through rigorous mathematical analysis that the 2-approximation algorithm is able to return a solution with approximate guarantees. Finally, we obtained results rigorously through experiments and present the analysis in the form of confidence interval plots and tables.

The remainder of this paper is organized as follows. Section II discusses the related work. Section III introduces the mathematical model and sets the objective function. The proposed algorithm is presented in detail in Section IV. Section V shows the evaluation of the results, and Section VI summarizes the entire work.

II. Related Work

In recent years, the quality of service issues in microservices-based online applications have attracted attention. Specifically, related research mainly includes service deployment, service chain deployment, request routing, and joint optimization of service deployment and request routing.

A. Service Deployment

In service deployment, a key challenge is optimizing service instance placement within network constraints. Various solutions have been proposed to address this challenge. Jiao et al. [22] employed a distributed deployment method for deploying C4ISR business processes across multiple data centers, yielding high-quality C4ISR business deployment schemes. Sun et al. [23] used a quality-of-service evaluation mechanism and a dynamic normal distribution selection method for service deployment, effectively reducing service response times and improving the average utilization rate of computing resources. Applegate et al. [24] proposed an intelligent content placement method and formulated the placement problem as a mixed integer program. They also analyzed the NP-hardness of the problem. Bunyakitanon et al. [10] designed a hierarchical Reinforcement Learning (RL) approach for orchestrating the dynamic placement of virtual network functions in cloud and edge 5 G environments. Fadda et al. [25] proposed an approach for supporting the deployment of microservices in multi-cloud environments focusing. Zhang et al. [26] considered the coupling relationship between edge servers and base stations, as well as between servers and services, addressing the problem using clustering and nonlinear programming algorithms. Karabey Aksakall et al. [27] introduced a tool named Micro-IDE, specifically designed for implementing and evaluating the effectiveness of microservice deployment. However, the current approaches are suboptimal for service deployment as they concentrate on individual services and overlook their interdependencies.

B. Service Chain Deployment

The evolving Internet sees user requests structured in a chain-like format. Single service deployment schemes fall short, and the emergence of service chains introduces new challenges. Various efforts are addressing the service chain deployment problem. Luo et al. [28] introduced a heuristic algorithm using Viterbi dynamic programming. This method estimates the occupancy of bottleneck resources for each parallel SFC, resulting in a near-optimal solution. Farkiani et al. [15] studied the deployment and reconfiguration of a set of chains with different priorities. Xiao et al. [29] proposed an adaptive deep RL approach that automatically deploys SFCs for requests with different QoS requirements. Han et al. [30] designed a service function chain deployment method based on network flow theory to reduce the average propagation delay of service flows. Bunyakitanon

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

et al. [31] present an adapted RL virtual network function performance prediction module for autonomous VNF placement. It leverages end-to-end service-level performance predictions for placing VNFs. Zhang et al. [32] proposed Dapper, a framework for deploying SFCs in the programmable data plane using deep reinforcement learning with graph convolutional network. Fu et al. [16] proposed a dynamic resource scaling framework based on streaming data analysis to address the service placement problem. Zhang et al. [33] optimized system performance using a time-difference learning mechanism for determining the optimal task assignment strategy in each decision phase. The cited literature has made substantial progress in service deployment but often treats it as an isolated module, overlooking communication between microservice instances.

C. Request Routing

The research on request routing has lasted for many years. Relevant researchers try to optimize service scheduling, time cost and response delay. During this period, some methods have been proposed and verified. Cong et al. [17] proposed an efficient personal-aware request scheduling scheme that maximized cloud providers' profits under the constraint of user satisfaction. Wang et al. [34] analyzed the optimal expected response time of each queue and proposed a multi-queue request scheduling algorithm framework, which effectively reduced the system response time. Tu et al. [18] designed two efficient request routing algorithms for offline and online scenarios, taking into account both tenant isolation and resource constraints. Zu et al. [35] proposed a model based on Nash negotiation through the method of cooperative game to jointly optimize delay and throughput and effectively reduce service delay. Chen et al. [36] proposed a Multi-Buffer Deep Deterministic Policy Gradient (MB_DDPG) algorithm to provide a more optimal scheduling solution. The above work focuses on the routing, scheduling, and distribution of user requests, but ignores the impact of service deployment on service quality.

D. Joint Optimization of Deployment and Routing

In recent years, there has been some work on optimizing service deployment and request routing. Bi et al. [37] developed a deep RL algorithm for multi-objective SFC placement and routing problems, leading to a significant reduction in service response delay. The work [38] studied the joint optimization problem of service deployment and request routing in MEC networks. However, user mobility over time and the long-term optimization of system performance are not considered in this way. Ren et al. [20] formulated adaptive Request Scheduling (RS) and cooperative Service Caching (SC) as partially observable Markov decision process problems. On this basis, an online algorithm based on deep RL was designed to improve the service delay reduction rate and the hit rate. Huang et al. [39] developed an approximation algorithm based on the Markov approximation technique for the joint service deployment and traffic scheduling (SUPER) problem to balance the cost and service quality. Although literature [20] and [39] considered service placement and request routing, they ignored the propagation delay in the routing process and perform poorly in reducing response delay.

TABLE I
NOTATION AND TERMINOLOGY

Notation	Definition
V, v	the set of edge nodes
C_v	number of CPU cores in edge node v
μ	the ability of a single core to process user requests
D	network propagation delay matrix
$d(v_1, v_2)$	propagation delay between edge nodes v_1 and v_2
M, m	the set of microservice types and microservices
F, f	the set of user request and user requests
M_f	the user requests a set of microservices contained in f
λ_f	the arrival rate of user request f
D_f^{max}	the maximum tolerable delay for the user request f
N_v^m	the deployment decision of microservice m on edge node v, integer variable
$P(m\|m_i, v\|v_i)$	probability that the request is routed to microservice m in edge node v after edge node v_i completes microservice m_i
λ_v^m	the user request arrival rate of microservice m in edge node v
$\overline{W_v^m}$	the processing delay of user requests through microservice m in edge node v
$L(f), f^i$	request the routing path collection and routing path
D_f	delay in responding to user requests
η_F	request success rate

Yu et al. [40] studied the optimization problem of microservice instance placement and request routing with an efficient heuristic algorithm. Another similar work [41] jointly optimise service placement and request scheduling by proposing a dual time scale framework. While the above study optimizes the joint optimization problem in some extent, the strong coupling of service deployment and request routing is weakened. In contrast, this paper fills the gap in this area to some extent by considering the complex communication between microservices.

III. PROBLEM FORMULATION

In this section, we introduce a formal set of models for microservice deployment and request routing, along with a detailed explanation of their parameters. We also provide a summary of the primary notations used in this paper and their corresponding physical meanings in Table I.

A. System Models

We model the MEC network, which describes the network structure and the propagating links of the edge nodes. Let $I = (V,\ D)$ denote an edge computing network with different geographical locations. In this network, $V = \{v_1,\ v_2,\ \ldots,\ v_{|V|}\}$ constitutes the set of edge nodes. Each edge node $v \in V$ comprises a base station responsible for communication and data propagation with neighboring edge nodes, as well as an edge server. The edge server provides essential service resources for deploying microservice instances and handling user requests. It is important to note that the number of CPU cores for each edge node, denoted as C_v, determines its service resource limit. The

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

processing capacity of a single core for user requests is represented by μ. Deploying microservice instances on a single core enhances isolation, ensuring that the workload of one service instance does not negatively impact others. This approach also streamlines the management and monitoring of service resource usage. Accordingly, this paper stipulates that a single CPU core can host only one microservice instance [19], [40]. Furthermore, a user's request may span different edge nodes due to geographic changes. These edge nodes cater to a variety of locations such as business centers, residential communities, and smart factories, each having distinct microservice deployment requirements. In this study, we define the coverage range of edge nodes to be between 200 to 400 meters [42]. User requests adhere to the principle of accessing the nearest edge node.

The network propagation delay matrix between edge nodes is denoted by D. It is determined by the respective locations of edge nodes and defines the request propagation time between two nodes. For example, if $D(v_1, v_2) = 3$, it implies a propagation delay of 3 milliseconds (ms) from node v_1 to v_2. Network propagation delay plays a crucial role in determining the response time of user service requests. Additionally, the network delay for request propagation within the same edge node is negligible, denoted as $d(v_1, v_1) = 0$. To further enrich our analysis of user service response delay, this paper integrates the $M/M/1$ queuing network into the system model, a topic discussed in detail in Section IV-A.

B. Microservice Models and User Request Models

Microservices are business requirements that are broken down into fine-grained services, each targeting a single user business. We use the notation $M = \{m_1,\ m_2,\ \ldots,\ m_{|M|}\}$ to denote the set of microservice types, which contains a variety of delay-sensitive microservices. For simplicity, this paper models user requests as chained structures. The set of user requests in MEC networks is denoted by $F = \{f_1,\ f_2,\ \ldots,\ f_{|F|}\}$, where the user requests f are chained structures, e.g., a chain of user requests for online payments is processed by four microservices (user information, balance inquiry, user payment, and balance update). Each user request is invoked in a fixed logical order as a microservice set $M_f = \{m_1,\ m_2,\ \ldots,\ m_{|M_f|}\}$. The user request arrival rate is denoted by the symbol λ_f, which is non-negative and follows a Poisson distribution. In addition, since all user request chains are delay sensitive, the symbol $D_f^{\max}$ is defined as the maximum tolerable delay for the user request f.

C. Decision Variables

The main objective of this paper is to optimize the microservice deployment as well as the routing scheme through decision variables. We need to make a decision for each edge node which microservices it should deploy to satisfy user request delay minimization. The decision to deploy microservices is represented as an integer variable $N_v^m \in \{0,\ \ldots,\ C_v\}$, which indicates the number of microservices m deployed (greater than or equal to 1) or not deployed ($= 0$). However, the computational resources of the edge nodes are significantly limited compared to the physical server clusters in the cloud data center. Hence, we have the following computational capacity limitations.

$$\forall v \in V,\ \textstyle\sum_{m\in M} N_v^m \leqslant C_v \tag{1}$$

If the current edge node is unable to handle the entire user request, it can route some of the user requests to nearby nodes. Request routing allows us to fully utilize computing resources among edge nodes, effectively reducing service delay and improving service quality. The request routing variable is $P(m|m_i,\ v|v_i) \in [0,\ 1]$, which denotes the probability of routing this user request flow to m in edge node v after edge node v_i completes microservice m_i. Obviously, edge nodes can only route user requests to those nodes where microservice m is deployed, and the total requests for each microservice m in each user request must be fully served, so we have the following constraints.

$$\forall m_i,\ \forall m,\ \forall v_i \quad \sum_{v\in V} P(m|m_i,\ v|v_i) = 1 \tag{2}$$

$$\forall v,\ \forall m,\ \forall f \quad P(m|m_i,\ v|v_i) \leqslant \frac{N_v^m}{C_v} \tag{3}$$

Since different user requests may share instances of microservices at the same edge node, request flows can be merged in the queuing network according to Burke's theorem and the linear additivity of Poisson flows. Defining the variable λ_v^m to denote the total user request reach rate at instances of microservice m in edge node v, for any $m \in M$ we have:

$$\forall m \in M,\ \forall v \in V,\ \lambda_v^m = \sum_{f\in F,\ m_i\in M\setminus m} \sum_{v_i\in V} \frac{P(m|m_i,\ v|v_i)}{\sum_{f\in F} P(m|m_i,\ v|v_i)} \lambda_{v_i}^{m_i} \tag{4}$$

Additionally, for network stability, we define that user requests routed should not exceed the service capacity of each microservice deployed on the edge node v, as follows:

$$\forall m \in M,\ \forall v \in V,\ \lambda_v^m \leqslant N_v^m \mu \tag{5}$$

Specifically, microservices need to be allocated adequate computing resources to handle incoming user requests into the system.

An example is provided for better comprehension. Let's consider two user requests: f_1 and f_2. We denote the set of user requests as $F = \{f_1,\ f_2\}$, and each user request corresponds to a processing order, that is:

$$f_1 : m_1 \to m_2 \to m_3 \to m_5$$

$$f_2 : m_3 \to m_4 \to m_1$$

To handle the aforementioned user requests, five microservices are required, denoted as $M = \{m_1,\ m_2,\ m_3,\ m_4,\ m_5\}$. We assume that the MEC scenario involves four edge nodes, i.e., $V = \{v_1,\ v_2,\ v_3,\ v_4\}$. Fig. 1 illustrates the topology of the multi-edge network with specific propagation delays between the edge nodes, represented by the propagation delay matrix D. The microservices are deployed on each edge node, and it should be noted that at least one instance of each microservice is deployed in the network. Furthermore, Fig. 1 illustrates the routing path of user request f_1 and how it is assigned to the microservice instance. For instance, when the request chain f_1 transitions from microservice m_2 on edge node v_1 to microservice m_3, the routing paths from m_2 to m_3 include $v_1 \to v_2$ and $v_1 \to v_3$. As a result, the routing path for user request

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

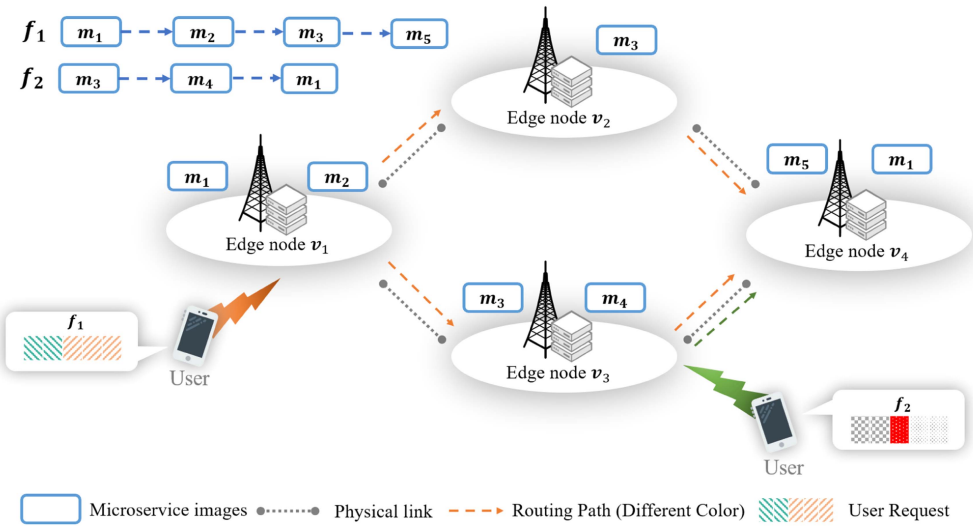


Fig. 1. Example of the mobile edge computing network. Microservice deployment and request routing policies are constrained by the computing resources of the edge node servers.

f_1 can be determined as $P(m_3|m_2, v_2|v_1) = 1$, or it could be expressed as a set of probabilities, e.g., $P(m_3|m_2, v_2|v_1) = 0.2$ and $P(m_3|m_2, v_3|v_1) = 0.8$.

IV. Algorithm Design and Analysis

MEC networks with multiple edge nodes aim to efficiently provide network services to users through effective microservice deployment and request routing. In this section, we innovatively build the well-established response delay minimisation problem based on queuing networks. Furthermore, we establish the NP-Hard nature of this problem. To tackle this challenge, we propose the Approximate Elastic Stretching algorithm for Joint Deployment and Routing (AES-JDR). AES-JDR is a 2-approximation algorithm designed to minimize the total response delay and enhance the request success rate. It achieves this by optimizing the request queue delay through fine-grained instance deployment. Additionally, it calculates the request propagation probability at the instance level, further enhancing request processing efficiency. Finally, we provide a comprehensive mathematical proof to ensure the performance reliability of the AES-JDR algorithm.

A. Optimization Objectives

The optimization goal that we are most interested in is delay, which determines the quality of service for processing user requests. In the mobile edge computing network described in this paper, the delay has two main components: the processing delay and the propagation delay.

Processing Delay: The time cost of a user request passing through the microservice m is represented by the auxiliary variable $\overline{W_v^m}$. According to the First Come First Serve (FCFS) queuing theory and the Processor Sharing (PS) principle, we have:

$$\forall m \in M,\ \forall v \in V,\ \overline{W_v^m} = \frac{1}{N_v^m \mu - \lambda_v^m} \tag{6}$$

Propagation Delay: To properly analyze the response delay of user requests, in addition to the sojourn delay of user requests, there is a request routing between edge nodes for user requests, so the network propagation delay between nodes needs to be considered. Define $L(f) = \{f^1,\ f^2,\ \ldots,\ f^{|L(f)|}\}$ to denote the set of user request $f \in F$ routing paths that follows the order of the microservice contained in M_f. The path $f^i = \{v_{m_1}^{f^i}, v_{m_2}^{f^i},\ \ldots,\ v_{m_{|M_f|}}^{f^i}\}$ of the ith request route contains the edge nodes passed under this path. The propagation delay under this path is:

$$\sum_{j=1}^{|f^i|-1} d(v_j^{f^i},\ v_{j+1}^{f^i}) \tag{7}$$

Defining D_f as the user request response delay, we have the following equation:

$$D_f = \sum_{k=1}^{|L(f)|}((\prod_{i=1}^{|M_F|-1} p(m_{i+1}|m_i,\ v_{i+1}|v_i))(\sum_{n=1}^{|f^i|} \overline{W_{f_n^i}^{m_n}} + \sum_{j=1}^{|f^i|-1} d(v_j^{f^i},\ v_{j+1}^{f^i}))) \tag{8}$$

Where, the expression $f_n^i = v_{m_n}^{f^i}$ denotes that the n th element in path f^i, representing microservice m_n, is executed on node $v_{m_n}^{f^i}$.

We minimize response delay by choosing the number and location of microservice instances and optimizing request routing. The symbol D_f is defined as the delay incurred to process a user request. In summary, this problem can be formulated as the Response Delay Minimization Problem (RDMP):

$$\min \quad D_f \tag{9}$$

$$s.t. \begin{cases} (1) - (5) \\ N_v^m \in \{0,\ \ldots,\ C_v\} \\ P(m|m_i,\ v|v_i) \in [0,\ 1] \end{cases} \tag{10}$$

By solving the above problem, we can get the response delay for each user request. The results must be evaluated using uniform criteria. Different types of user requests have different maximum tolerable delays. Therefore, we focus on the relationship between the actual user request delay D_f and the maximum tolerable delay $D_f^{\max}$, i.e., the user request success rate. Evaluating the resulting performance based on the success rate of large-scale user requests is a common approach adopted

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

by mobile edge computing operators.

$$\eta_F = \frac{|F_{suc}(D_f \leqslant D_f^{\max})|}{|F|} \tag{11}$$

In (11), η_F is used to denote the percentage of user requests that are successfully serviced within the maximum tolerable delay, $D_f^{\max}$ denotes the maximum tolerable delay for the user request f, and $|F|$ denotes the total number of arriving user requests, and the set F_{suc} represents the successful user requests.

B. Hardness Proof

The problem of minimizing delay, transformed into an MILP problem [43] by integer deployment decisions, is proven to be NP-hard as demonstrated by the following theorem:

Theorem 1. The delay minimization problem is NP-hard.

Proof. First, it is necessary to introduce the concept of the Uncapped Facility Location (UFL) problem.

$$\min \sum_{j \in Q} \sum_{i \in N} c_{ij} x_{ij} + \sum_{i \in N} f_i y_i \tag{12}$$

$$s.t. \begin{cases} \sum_{i \in N} x_{ij} = 1 \quad \forall j \in Q \\ x_{ij} \leqslant y_i \\ x_{ij} \geqslant 0 \quad \forall i \in N, j \in Q \\ x_{ij} \in [0,1], y_i \in \{0,1\} \quad \forall i \in N, j \in Q \end{cases} \tag{13}$$

In a typical unrestricted facility location problem, we are given a set of facilities $i \in N$ with corresponding facility opening costs f_i, and a set of customers $j \in Q$ with demands. The objective is to determine which facilities to open (represented by decision variable y_i) and how to allocate customers to the open facilities (represented by decision variable x_{ij}) in order to minimize the total cost of facility opening ($\sum_{i \in N} f_i y_i$) and customer allocation ($\min \sum_{j \in Q} \sum_{i \in N} c_{ij} x_{ij}$), where c_{ij} represents the cost of allocating customer j to facility i.

The UFL problem is a well-known NP-hard problem. By symbolically substituting the microservice placement decision N_v^m with the facility opening decision y_i, and the request routing decision $\widetilde{P}(m|m_i, v|v_i)$ with the allocation decision x_{ij}, the RDMP can be transformed into a simplified UFL problem. Hence, it can be concluded that the RDMP is also an NP-hard problem. ■

C. Algorithm Design

Given that response delay minimization is an NP-hard problem, directly obtaining an optimal solution using a solver becomes unfeasible when dealing with large-scale edge networks. Extensive research has been conducted on the MILP problem, yielding numerous excellent results. A fundamental approach involves initially relaxing the integer constraints of the MILP problem to attain an optimal fractional solution. This fractional solution is then transformed into an integer solution, taking into account the resource constraints specific to the problem.

We have devised an approximate elastic stretching algorithm for joint deployment and routing employing a rounding technique to approximate the optimal fractional solution. This is essential as the presence of integer variables significantly complicates the problem. Once fractional deployment decisions are made, the remaining request routing problem can be optimally resolved using classical linear programming techniques such as the simplex method. Consequently, obtaining a fractional deployment solution is of paramount importance. One approach involves relaxing integer variables to achieve the optimal fractional solution, followed by applying a random independent

Algorithm 1. Approximate Elastic Stretching Algorithm for Joint Deployment and Routing

Input:
The MEC network I, the maximum tolerable delay $D_f^{\max}$, the propagation delay matrix D, the number of edge node CPU cores C_v, and the single core computing capability μ.
Output:
The number of service deployments $\widetilde{N_v^m}$ and the forwarding probability $\tilde{P}$.

Get user request set F and request arrival rate λ_f;
Solve the linear relaxation of RDMP problem to obtain $(N_v^{m^*},\ P^*(m|m_i,\ v|v_i))$ optimal solution.
for $v \in V$ **do**
  $flag \leftarrow 0$;
  for $m \in M$ **do**
    $lable \leftarrow round(N_v^{m^*})$;
    if $lable \geq N_v^{m*}$ **then**
      $flag \leftarrow flag + (lable - N_v^{m*})$
    else
      $flag \leftarrow falg - (N_v^{m*} - lable)$
    end if
    if $flag \geq 0.999$ **then**
      $flag \leftarrow flag - 2 * (lable - N_v^{m*})$
      $\widetilde{N_v^m} \leftarrow \lfloor N_v^{m*} \rfloor$;
    else
      $\widetilde{N_v^m} \leftarrow round(N_v^{m^*})$;
    end if
  end for
end for
Deployment probability $X_v^m \leftarrow \frac{N_v^{m^*}}{C_v}$;
for $f \in F$ **do**
  $\Omega_i\{v \in V\} \cap \{v|N_v^{m_i} > 0,\ m_i \in M_f\}$;
  for $m_i \in M_f$ **do**
    for $v_i \in \Omega_i$ **do**
      Routing probability
      $\tilde{P}(m|m_i,\ v|v_i) \leftarrow \frac{P^*(m|m_i,\ v|v_i)}{X_{v_i}^{m_i}}$;
    end for
  end for
end for
$\tilde{P}_{v_i}^{m_i}$ is the set of $\tilde{P}(m|m_i,\ v|v_i)$;
for $m \in M,\ v \in V$ **do**
  $P \leftarrow sum(\tilde{P}_{v_i}^{m_i})$
  Elements in $\tilde{P}_{v_i}^{m_i}$ are adaptively scaled according to $\frac{1}{P}$;
  $\tilde{P}_{v_i}^{m_i}$ add to request routing probability matrix $\tilde{P}$;
end for
return $(\widetilde{N_v^m},\ \tilde{P})$;

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

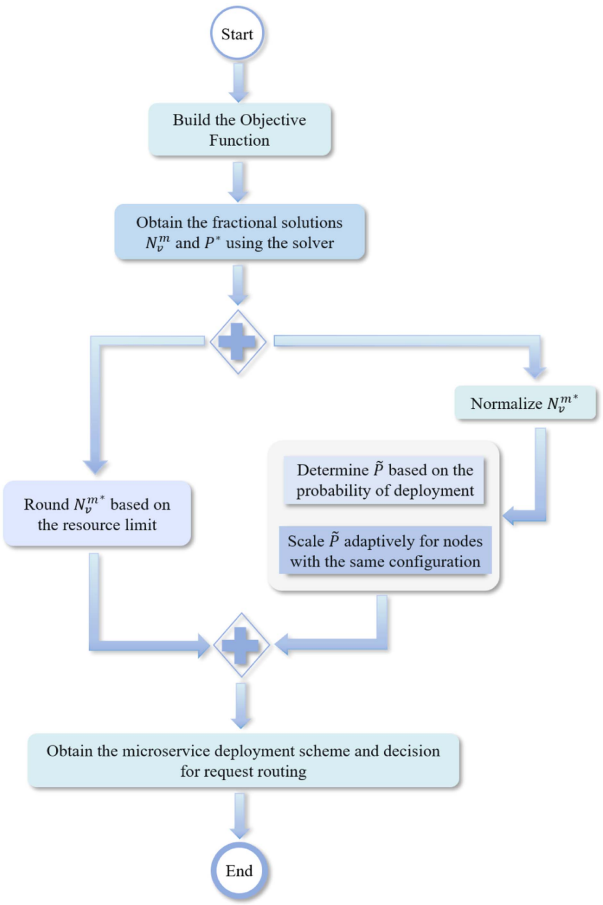


Fig. 2. Business process model and notation (BPMN) diagram of AES-JDR algorithm.

rounding scheme to derive a suboptimal integer solution. However, this rounding scheme may produce infeasible solutions in certain specific cases (e.g., rounding up all variables, potentially violating edge node computing resource capacity constraints). To address such issues, we designed a rounding scheme, summarized in Algorithm 1, and also provided a business process model and symbolic diagram for the AES-JDR algorithm, as shown in Fig. 2.

The AES-JDR algorithm first solves the linear relaxation of the problem of minimizing response delay (Line 2). This means that it changes the variable N_v^m to be a fraction rather than an integer. Since the objective and constraint conditions of the problem are linear, a linear solver can be used to obtain the optimal solution in polynomial time. We use $(N_v^{m^*}, P^*)$ to represent the optimal solution value. The next step is to adjust $N_v^{m^*}$ to get the integer solution $\widetilde{N_v^m}$. For each edge node and microservice combination $N_v^{m^*}$, we need to understand that the computing resource constraints of the edge node must be met when rounding. First, a tag value $flag$ is defined for each edge node and initialized to 0 (Line 4). And then, $N_v^{m^*}$ is rounded to the integer $lable$. For $lable \geqslant N_v^{m*}$, $flag$ adds $lable - N_v^{m*}$, and conversely, $flag$ subtracts $N_v^{m*} - lable$ (Line 6). Further, to meet the computational resource limit, when $flag \geqslant 0.999$, update $flag = flag - 2 * (lable - N_v^{m*})$ and round down N_v^{m*}. Otherwise, N_v^{m*} will be rounded normally to obtain the suboptimal integer deployment decision $\widetilde{N_v^m}$ (Lines 7-18).

After the integer adjustment of the deployment decision, we need to make adaptive adjustments to the request routing probability. In particular, the sum of the request routing probabilities of microservice m in edge node v should always be 1. The algorithm first normalizes N_v^{m*} to obtain the deployment probability of microservice m at edge node v, i.e., $X_v^m = \frac{N_v^{m*}}{C_v}$ (Line 19). Then, the deployment probability X_v^m is used to adjust the request routing (Lines 20-31). For microservice m included in user request f, we define Ω_i as the set of edge nodes deployed by microservice m (Line 21) and use this setting to calculate the routing probability of user requests. The routing probability $\tilde{P}$ of user request depends on normalized $N_v^{m^*}$ and fraction value P^*. The edge node with larger $\tilde{P}$ is given higher probability. It means that more user requests are diverted to this node to process (lines 21-27). The definition of $\widetilde{P}_{v_i}^{m_i}$ is the set of transition out probabilities of microservice m_i at edge node v_i, which is summed and recorded as P. According to the ratio of P to 1, each element in array $\widetilde{P}_{v_i}^{m_i}$ is scaled adaptively, and the request routing probability matrix $\widetilde{P}$ is obtained (Lines 28-31). Then, we verify the quality of the solution returned by the AES-JDR algorithm. We have the following theorem.

Theorem 2. The AES-JDR algorithm can efficiently route each user request as the number of edge node CPU cores and service demand increase.

Proof. First, we need to normalize N_v^{m*} to obtain the probability of deployment of microservice m on edge nodes, which means $N_v^{m*} = \frac{N_v^{m*}}{C_v}$. For any given user request $f \in F$, the request routing variable $P^*(m|m_i,\ v|v_i)$ is adjusted in two cases:

1) Microservices are deployed on one or more of the edge nodes. $(\Omega_i \neq \varnothing)$

2) Microservices are not deployed on the edge node. $(\Omega_i = \varnothing)$

The probability of user requests being routed is:

$$\begin{aligned} &Pr[\tilde{P}(m|m_i,\ v|v_i)] \\ &= Pr[\tilde{P}(m|m_i,\ v|v_i)|\widetilde{N_v^m} > 0]Pr[\widetilde{N_v^m} > 0] \\ &\quad + Pr[\tilde{P}(m|m_i,\ v|v_i)|\widetilde{N_v^m} = 0]Pr[\widetilde{N_v^m} = 0] \\ &= \frac{P^*(m|m_i,\ v|v_i)}{N_v^{m^*}} N_v^{m^*} = P^*(m|m_i,\ v|v_i) \end{aligned} \tag{14}$$

Obviously, the right side of (14) consists of two parts, corresponding to the two cases of request routing. First, according to the basic definition of conditional probability and $\widetilde{N_v^m}$ value, we can get $\widetilde{P}(m|m_i,\ v|v_i)$ rounding decisions. This is where the first part of the right-hand side of (14) comes from. The second case corresponds to the second part on the right-hand side of the formula, we can easily obtain the result $Pr[\tilde{P}(m|m_i,\ v|v_i)|\widetilde{N_v^m} = 0]Pr[\widetilde{N_v^m} = 0] = 0$. ■

D. The Approximation Ratio

The user request response delay consists of the microservice linger delay and the propagation delay, which is obtained from the objective function.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

According to (7) and the preceding propagation delay analysis, we can calculate the link propagation delay needed for a single routing path when a user request enters the system for processing. In addition, we account for the routing probability to compute the total link propagation delay, which is expressed as follows:

$$\sum_{k=1}^{|L(f)|}\left(\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\left(\sum_{j=1}^{|f^i|-1}d(v_j^{f^i},\ v_{j+1}^{f^i})\right)\right) \tag{15}$$

Obviously, the propagation delay of user requests is mainly determined by the set $L(f)$ of user request $f \in F$ routing paths.

Next, we analyze the processing delay of microservices. First, according to the queuing theory, we can get the dwell time $\overline{W^m}$ of the user request on each microservice:

$$\overline{W^m} = \frac{1}{N^m\mu - \lambda_f} \tag{16}$$

The value of $\overline{W^m}$ is determined by the number N^m of deployments of the microservice m.

Combined with the objective function of the formula, it is further obtained that the total service linger delay in the user request response delay is:

$$\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\overline{W^m} \tag{17}$$

We assume that $L_{\min}$ and $L_{\max}$ represent the minimum and maximum sets of user request routes, respectively; the optimal service deployment is denoted by N^{m^*}; and the optimal dwell time delay is $\overline{W^{m^*}}$. At the same time, we define OPT^* as the optimal service response delay for user requests:

$$\begin{aligned}OPT^* = &\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\overline{W^{m^*}}\\ &+\sum_{k=1}^{L_{\min}}\left(\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\right.\\ &\left.\left(\sum_{j=1}^{|f^i|-1}d(v_j^{f^i},\ v_{j+1}^{f^i})\right)\right)\end{aligned} \tag{18}$$

The target value of this algorithm can be expressed as:

$$\begin{aligned}Target \leqslant &\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\overline{W^m}\\ &+\sum_{k=1}^{L_{\max}}\left(\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\right.\\ &\left.\left(\sum_{j=1}^{|f^i|-1}d(v_j^{f^i},\ v_{j+1}^{f^i})\right)\right)\end{aligned} \tag{19}$$

By taking the ratio of formulas (18) and (19) and simplifying, we can derive the approximate ratio for the AES-JDR algorithm:

$$\begin{aligned}\frac{Target}{OPT^*} \leqslant &\left[\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\overline{W^{m^*}}+\right.\\ &\left.\sum_{k=1}^{L_{\max}}\left(\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\right.\right.\\ &\left.\left.\left(\sum_{j=1}^{|f^i|-1}d(v_j^{f^i},\ v_{j+1}^{f^i})\right)\right)\right]\Big/\\ &\left[\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\overline{W^{m^*}}+\right.\\ &\left.\sum_{k=1}^{L_{\min}}\left(\left(\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)\right)\right.\right.\\ &\left.\left.\left(\sum_{j=1}^{|f^i|-1}d(v_j^{f^i},\ v_{j+1}^{f^i})\right)\right)\right]\end{aligned} \tag{20}$$

$$\frac{Target}{OPT^*} < \frac{\overline{W^m}}{\overline{W^{m^*}}} + \frac{\sum_{k=1}^{L_{\max}}\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)}{\sum_{k=1}^{L_{\min}}\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i))} \tag{21}$$

The approximate ratio in formula (21) can be expressed as follows by assuming: $\varepsilon = \frac{\overline{W^m}}{\overline{W^{m^*}}}$ and $K = \frac{\sum_{k=1}^{L_{\max}}\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i)}{\sum_{k=1}^{L_{\min}}\prod_{i=1}^{|M_f|-1}P(m_{i+1}|m_i,\ v_{i+1}|v_i))}$.

$$ratio < k + \varepsilon \tag{22}$$

The ratio is less than 2 when K and ε is equal to 1.

V. Performance Evaluation

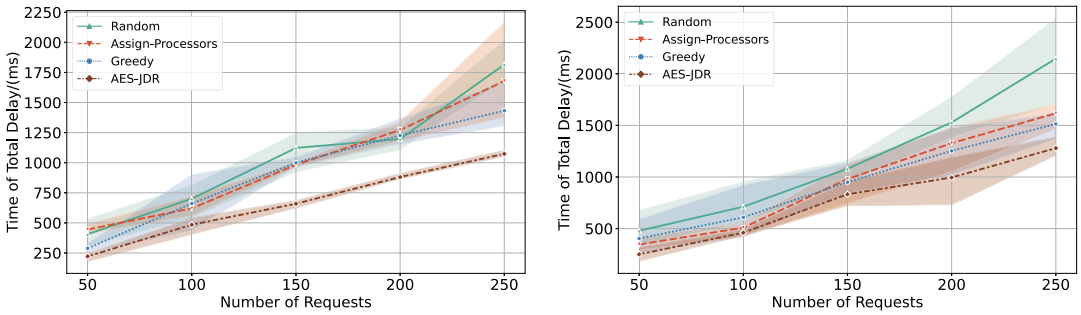
In this section, we evaluate the performance of the AES-JDR algorithm through extensive trace-driven simulations. For our experiments, we utilize the cluster-trace-microservices-v2021 and cluster-trace-microarchitecture-v2022 datasets, publicly available from Alibaba [5]. These datasets are processed to obtain the necessary information. To ensure the reliability and stability of the experiments, all the results given in this paper are the average of 500 independent experiments. Our experimental parameter settings are based on real-world data traces from MEC network scenarios. Additionally, this paper compare the AES-JDR algorithm with three baseline methods.

Random: Microservices are randomly deployed at each edge node, while user requests arriving on the MEC network are randomly routed to the appropriate edge node for processing.

Greedy: The algorithm attempts to deploy different types of microservices on an edge node because it is more likely to complete the arrival of the entire request flow on a single node, thus avoiding additional route propagation delays.

Assign-Processors [16]*:* The method takes the total amount of computing resources as a constraint and adds one core to the minimum number of cores required for each microservice to analyze the results. After several iterations, the deployment scheme that minimizes the processing delay is obtained.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.



(a) Long tail distribution (b) Random distribution

Fig. 3. Delay for different number of requests.

TABLE II
SUCCESS RATE FOR DIFFERENT NUMBER OF REQUESTS

Distribution	Number of User Requests	Success rate (%)			
		Random	Assign-Processors	Greedy	AES-JDR
Long tail	50	92	90	94	**100**
	100	87	89	92	**99**
	150	85	87	91	**98**
	200	85	86	90	**97**
	250	84	85	90	**97**
Random	50	94	98	96	**100**
	100	92	94	94	**99**
	150	91	94	93	**97**
	200	91	93	93	**96**
	250	86	92	92	**95**

In order to highlight the experimental results of the method AES-JDR in this paper, we show them in bold.

A. Experiment Settings

Physical Networks: We consider a connected service grid to simulate an MEC network containing 10 edge nodes. In addition, in our scenario, the edge server is local friendly, so the local execution delay is usually smaller than the route propagation delay between edge nodes, and a propagation delay matrix with dimension $10 * 10$ is used to represent the different propagation delay between each edge node. The computing resources in edge nodes are represented by the number of CPU cores. By default, each edge node contains 10 CPU cores for processing user requests. This means that up to 10 microservice instances can be deployed per edge node. At the same time, each core is defined to process different kinds of microservices at different rates.

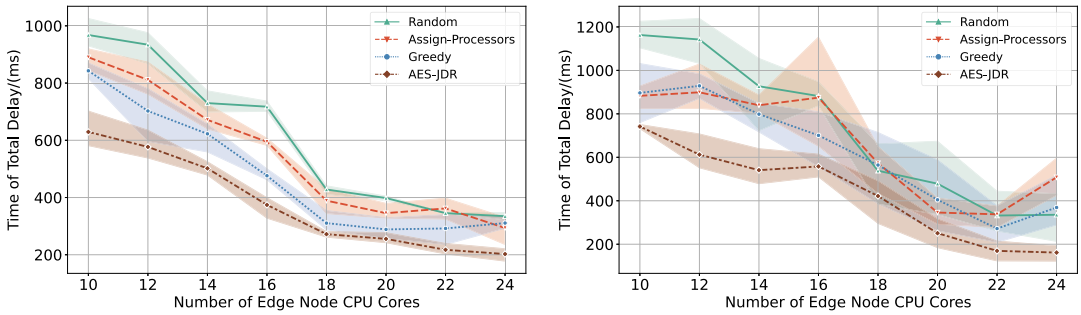
Microservices and Request Chains: Through the previous research work, the scale of commonly used microservices is between 10 to 40, including at least two commonly deployed microservice businesses. In our setup, the number of request chains ranges from 50 to 250 according to the characteristics of user requests. Each request chain traverses the corresponding microservice instance in a defined order. The length is set between 2 to 7 and the types of microservices are between 2 to 5 [5], [7]. It is important to note that the same microservice may exist in the same chain of requests. At the same time, the arrival rate of the user request chain is also an important parameter that affects the service response delay. Without loss of generality, the arrival rate is set to random distribution and long-tail distribution, so as to observe the effect of each parameter on the experimental results under different arrival rate distribution.

Experimental Indicators: We conducted over 500 independent experiments, systematically varying critical parameters within the MEC network to thoroughly evaluate the effectiveness and stability of our proposed methodology. Additionally, for ease of analysis, we also presented the results of the two critical experimental metrics, total response delay, and request success rate, in the form of confidence interval plots and tables.

B. The Effect of Number of User Requests

We begin by investigating the impact of the number of user requests on service response delay and request success rate for two arrival rate distributions. We consider different numbers of user requests: $[50, 100, 150, 200, 250]$. The results are presented in Fig. 3 and Table II. It is evident that with an increase in the number of user requests, the total service response delay gradually increases, and correspondingly, the request success rate gradually decreases. Nonetheless, the proposed AES-JDR algorithm consistently outperforms the baseline algorithms. Significantly, as illustrated in Fig. 3(a), our algorithm demonstrates exceptional stability when the number of user requests surpasses

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.



(a) Long tail distribution (b) Random distribution

Fig. 4. Delay for different number of edge node CPU cores.

TABLE III
SUCCESS RATE FOR DIFFERENT NUMBER OF EDGE NODE CPU CORES

Distribution	Number of edge node CPU cores	Success rate (%)			
		Random	Assign-Processors	Greedy	AES-JDR
Long tail	10	62	66	68	**86**
	12	72	74	74	**90**
	14	76	80	82	**90**
	16	80	82	84	**94**
	18	82	82	86	**98**
	20	84	88	86	**100**
	22	88	90	88	**100**
	24	92	92	94	**100**
Random	10	60	66	64	**78**
	12	64	66	70	**82**
	14	69	70	72	**88**
	16	70	64	78	**90**
	18	84	82	80	**92**
	20	86	86	82	**94**
	22	90	88	88	**98**
	24	91	86	87	**100**

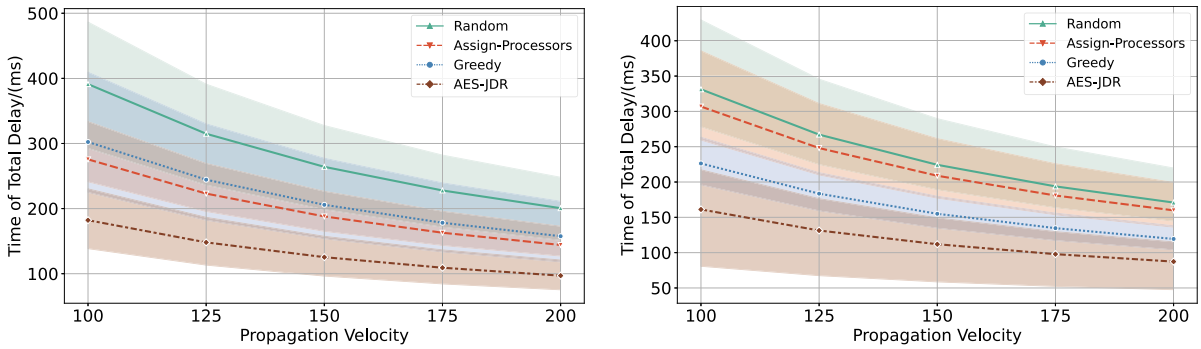
In order to highlight the experimental results of the method AES-JDR in this paper, we show them in bold.

150, outperforming the three baseline algorithms. This intriguing observation implies that AES-JDR is particularly well-suited for scenarios involving a high volume of user requests. Furthermore, AES-JDR maintains optimal performance in a random distribution. Table II illustrates that under both arrival rate distributions, the request success rate steadily declines as the number of user requests increases. This decline is attributed to the heightened load on the MEC network caused by the increase in user requests, resulting in a degradation of network quality of service.

C. The Effect of the Number of Edge Node CPU Cores

In addition to discussing the impact of user request chain parameters, we also fixed the user request chain parameters in our experiments and set the number of CPU cores of the server to $[10, 12, 14, 16, 18, 20, 22, 24]$. On this basis, we evaluated the performance of the four algorithms. The results are shown in Fig. 4 and Table III. Compared with Fig. 3, Fig. 4 shows an overall decreasing trend in the total response delay, which is due to the fact that more CPU cores mean that the network is more capable of processing user requests per unit of time, which reduces the user request processing delay. Compared to the other three baseline algorithms, the algorithm in this paper is the first to show convergence characteristics in Fig. 4(a). This is due to the fact that the AES-JDR algorithm performs adaptive probabilistic routing of user requests, which better utilises the processing power of the CPU cores, effectively reduces the number of CPU cores required, and also brilliantly reduces the operator cost. An interesting phenomenon is that in Fig. 4(b), when the number of CPU cores is 16, the total response delay of Assign-Processors appears to increase significantly, this is because, Assign-Processors only focuses on the deployment of microservice instances, and when the number of CPU cores is increased, it will deploy the same type of instances in the same edge node which reduces the service delay. However, this increases the route propagation delay and causes the total response delay to rise. As the number of CPU cores increases and the computational power of the edge nodes is enhanced, the total response delay of Assign-Processors again shows a decreasing

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.



(a) Long tail distribution (b) Random distribution

Fig. 5. Delay for different propagation velocities.

TABLE IV
SUCCESS RATE FOR DIFFERENT PROPAGATION VELOCITIES

Distribution	Size of propagation velocities	Success rate (%)			
		Random	Assign-Processors	Greedy	AES-JDR
Long tail	100	62	76	74	**78**
	125	72	80	78	**90**
	150	74	80	78	**98**
	175	78	82	80	**100**
	200	84	92	90	**100**
Random	100	74	76	80	**82**
	125	74	80	82	**86**
	150	78	82	84	**94**
	175	80	84	84	**94**
	200	82	86	90	**94**

In order to highlight the experimental results of the method AES-JDR in this paper, we show them in bold.

trend. Compared to Assign-Processors, AES-JDR jointly optimises microservice instance deployment and request routing, and the total response delay is always optimal. In addition, as shown in Table III, we observe that AES-JDR consistently demonstrates a gradual increase and eventual convergence in request success rates. In contrast, Assign-Processors exhibits considerable volatility. This substantiates the superior stability and reliability of the algorithm proposed in this paper.

D. The Effect of Propagation Velocity

Next, to investigate the effect of different propagation velocities on the service response delay and request success rate, we set the range of propagation velocities to be $[100, 125, 150, 175, 200]$. In Fig. 5, with a constant user request, the service response delay tends to decrease as the propagation velocity increases. This relationship is intuitive. Higher propagation velocity leads to lower propagation delay for user requests when they are routed probabilistically. Consequently, this reduction in propagation delay results in a decrease in the total user request response delay across all four algorithms. Furthermore, the user request success rate is influenced by both the request propagation velocity and the total response delay. The detailed results are presented in Table IV. As observed in Table IV, AES-JDR consistently outperforms all three baseline algorithms for both arrival rate distributions as propagation velocities increase. Notably, AES-JDR achieves the highest and stable request success rates of 100% and 94% for the respective arrival rate distributions. This superiority is attributed to the joint optimization approach of the AES-JDR algorithm. It not only emphasizes the request routing decision but also optimizes the deployment of microservice instances, effectively reducing service processing delays and demonstrating superior performance in improving request success rates.

E. The Effect of Length of User Request

Finally, based on the literature [5], [7],we vary the request chain length over the range $[2, 3, 4, 5, 6, 7]$ with a random distribution of arrival rates to examine its effect on the results. The outcomes are presented in Fig. 6 and Table V. As illustrated in Fig. 6, the overall service response delay for all four algorithms tends to increase with longer request lengths. However, the AES-JDR algorithm consistently maintains the lowest total response delay, showcasing a significantly smaller increase compared to the baseline algorithm. This superiority stems from AES-JDR's optimization not only of microservice deployment location and quantity but also of user request routing. Unlike the baseline algorithm's random or greedy routing approach, AES-JDR employs adaptive probabilistic routing, enabling it to select user request routing paths more optimally. Table V presents the request success rate results, demonstrating that an increase in

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SUCCESS RATE OF DIFFERENT USER REQUEST LENGTHS

Distribution	Size of user request lengths	Success rate (%)			
		Random	Assign-Processors	Greedy	AES-JDR
Random	2	98	95	96	**100**
	3	95	94	95	**100**
	4	88	93	91	**97**
	5	87	91	91	**95**
	6	82	90	89	**94**
	7	76	87	91	**92**

In order to highlight the experimental results of the method AES-JDR in this paper, we show them in bold.

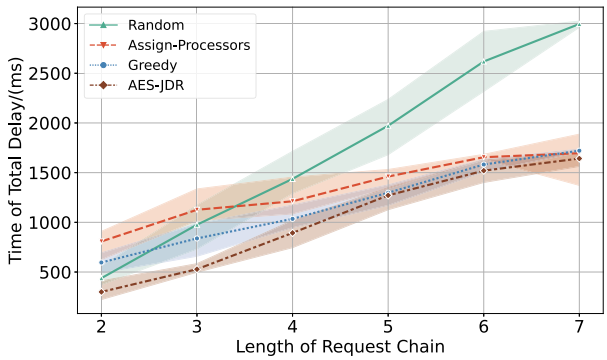


Fig. 6. Delay for different user request lengths.

request length imposes greater pressure on the network, resulting in a decline in the success rate of user requests. Nonetheless, the algorithm presented in this paper consistently outperforms in terms of success rate. Even with a chain length of 6, there is a notable success rate of up to 94%. These results underline AES-JDR's ability to offer superior quality of service to the network.

VI. CONCLUSION

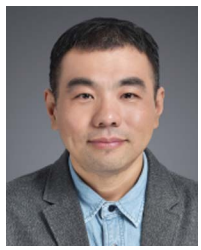
In this paper, we have studied the joint optimization of microservice deployment and request routing in the MEC scenario. Specifically, we consider constraints such as computing resources, request routing, and service capacity to minimize the response delay of user requests. We model it as an MILP problem and prove its hardness using an uncapacitated facility location problem. In addition, we propose an approximation algorithm AES-JDR based on the rounding idea to solve the microservice deployment and request routing problem in edge scenarios. At the same time, the performance of the algorithm is theoretically analyzed, and it is proved that the algorithm can guarantee the performance of the approximate result. Finally, through sufficient simulation experiments, the performance of the algorithm is evaluated in terms of total delay and success rate of user requests when the user requests are random distribution or long-tail distribution. The simulation results clearly demonstrate that AES-JDR can not only effectively utilize the computing resources of edge nodes, but also keep the service response delay to a minimum. Moreover, it exhibits notable stability, making it highly suitable for large-scale user request scenarios. In addition, the request success rate of the proposed algorithm is also excellent in different request distribution scenarios. For future research, we will consider multi-resource network environments and different network typologies, and hope to solve the dynamic request scheduling problem in a real microservice architecture.

REFERENCES

[1] A. Sill, "The design and architecture of microservices," *IEEE Cloud Comput.*, vol. 3, no. 5, pp. 76–80, Sep./Oct. 2016.
[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter, 2017.
[3] Dubbo TEAM, "Apach dubbo," 2023. [Online]. Available: http://dubbo.apache.org/
[4] Spring Team, "Spring cloud," 2023. [Online]. Available: http://springciould.cc/
[5] Alibaba, "Alibab cluster data," 2023. [Online]. Available: https://github.com/alibaba/clusterdata
[6] M. G. Khan, J. Taheri, A. Al-dulaimy, and A. Kassler, "PerfSim: A performance simulator for cloud native microservice chains," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1395–1413, Second Quarter, 2023.
[7] S. Luo et al., "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proc. ACM Symp. Cloud Comput.*, New York, NY, USA, 2021, pp. 412–426.
[8] B. Li et al., "READ: Robustness-oriented edge application deployment in edge computing environment," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1746–1759, May/Jun. 2022.
[9] B. Sonkoly, J. Czentye, M. Szalay, B. Németh, and L. Toka, "Survey on placement methods in the edge and beyond," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2590–2629, Fourth Quarter, 2021.
[10] M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "HELICON: Orchestrating low-latent & load-balanced virtual network functions," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 353–358.
[11] B. Wang, C. Wang, W. Huang, Y. Song, and X. Qin, "A survey and taxonomy on task offloading for edge-cloud computing," *IEEE Access*, vol. 8, pp. 186080–186101, 2020.
[12] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, Fourth Quarter, 2021.
[13] S. Wang, X. Li, Q. Z. Sheng, and A. Beheshti, "Performance analysis and optimization on scheduling stochastic cloud service requests: A survey," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 3587–3602, Sep. 2022.
[14] M. Hu, J. Luo, Y. Wang, and B. Veeravalli, "Practical resource provisioning and caching with dynamic resilience for cloud-based content distribution networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 8, pp. 2169–2179, Aug. 2014.
[15] B. Farkiani, B. Bakhshi, S. A. MirHassani, T. Wauters, B. Volckaert, and F. De Turck, "Prioritized deployment of dynamic service function chains," *IEEE/ACM Trans. Netw.*, vol. 29, no. 3, pp. 979–993, Jun. 2021.
[16] T. Z. J. Fu, J. Ding, R. T. B. Ma, M. Winslett, Y. Yang, and Z. Zhang, "DRS: Auto-scaling for real-time stream analytics," *IEEE/ACM Trans. Netw.*, vol. 25, no. 6, pp. 3338–3352, Dec. 2017.
[17] P. Cong, G. Xu, J. Zhou, M. Chen, T. Wei, and M. Qiu, "Personality- and value-aware scheduling of user requests in cloud for profit maximization," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1991–2004, Third Quarter, 2022.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.

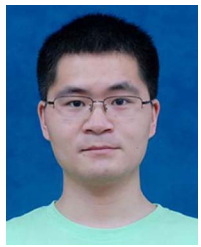
[18] H. Tu, G. Zhao, H. Xu, and X. Fang, "Tenant-grained request scheduling in software-defined cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4654–4671, Dec. 2022.
[19] Y. Hu, H. Wang, L. Wang, M. Hu, K. Peng, and B. Veeravalli, "Joint deployment and request routing for microservice call graphs in data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 11, pp. 2994–3011, Nov. 2023.
[20] D. Ren, X. Gui, and K. Zhang, "Adaptive request scheduling and service caching for MEC-assisted IoT networks: An online learning approach," *IEEE Internet of Things J.*, vol. 9, no. 18, pp. 17372–17386, Sep. 2022.
[21] Y. Wang, W. Shi, and M. Hu, "Virtual servers co-migration for mobile accesses: Online versus off-line," *IEEE Trans. Mobile Comput.*, vol. 14, no. 12, pp. 2576–2589, Dec. 2015.
[22] Z. Jiao, J. Zhang, P. Yao, L. Wan, and L. Ni, "Service deployment of C4ISR based on genetic simulated annealing algorithm," *IEEE Access*, vol. 8, pp. 65498–65512, 2020.
[23] H. Sun, S. Wang, F. Zhou, L. Yin, and M. Liu, "Dynamic deployment and scheduling strategy for dual-service pooling based hierarchical cloud service system in intelligent buildings," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 139–155, First Quarter, 2023.
[24] D. Applegate, A. Archer, V. Gopalakrishnan, S. Lee, and K. K. Ramakrishnan, "Optimal content placement for a large-scale VoD system," *IEEE/ACM Trans. Netw.*, vol. 24, no. 4, pp. 2114–2127, Aug. 2016.
[25] E. Fadda, P. Plebani, and M. Vitali, "Monitoring-aware optimal deployment for applications based on microservices," *IEEE Trans. Services Comput.*, vol. 14, no. 6, pp. 1849–1863, Nov./Dec. 2021.
[26] X. Zhang, Z. Li, C. Lai, and J. Zhang, "Joint edge server placement and service placement in mobile-edge computing," *IEEE Internet of Things J.*, vol. 9, no. 13, pp. 11261–11274, Jul. 2022.
[27] I. K. Aksakallı, T. Çelik, A. B. Can, and B. Tekinerdoğan, "Micro-IDE: A tool platform for generating efficient deployment alternatives based on microservices," *Softw.: Pract. Experience*, vol. 52, no. 7, pp. 1756–1782, 2022.
[28] J. Luo, J. Li, L. Jiao, and J. Cai, "On the effective parallelization and near-optimal deployment of service function chains," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1238–1255, May 2021.
[29] Y. Xiao et al., "NFVdeep: Adaptive online service function chain deployment with deep reinforcement learning," in *Proc. IEEE/ACM 27th Int. Symp. on Qual. of Serv.*, 2019, pp. 1–10.
[30] X. Han, X. Meng, Z. Yu, Q. Kang, and Y. Zhao, "A service function chain deployment method based on network flow theory for load balance in operator networks," *IEEE Access*, vol. 8, pp. 93187–93199, 2020.
[31] M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "End-to-end performance-based autonomous VNF placement with adopted reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 534–547, Jun. 2020.
[32] X. Zhang, L. Cui, F. P. Tso, Z. Li, and W. Jia, "Dapper: Deploying service function chains in the programmable data plane via deep reinforcement learning," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2532–2544, Jul./Aug. 2023.
[33] Y. Zhang, Z. Zhou, Z. Shi, L. Meng, and Z. Zhang, "Online scheduling optimization for DAG-based requests through reinforcement learning in collaboration edge networks," *IEEE Access*, vol. 8, pp. 72985–72996, 2020.
[34] S. Wang, X. Li, Q. Z. Sheng, R. Ruiz, J. Zhang, and A. Beheshti, "Multi-queue request scheduling for profit maximization in IaaS clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2838–2851, Nov. 2021.
[35] J. Zu, G. Hu, D. Peng, S. Xie, and W. Gao, "Fair scheduling and rate control for service function chain in NFV enabled data center," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 2975–2986, Sep. 2021.
[36] L. Chen et al., "IoT microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE Internet of Things J.*, vol. 8, no. 16, pp. 12610–12622, Aug. 2021.
[37] Y. Bi, C. C. Meixner, M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "Multi-objective deep reinforcement learning assisted service function chains placement," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4134–4150, Dec. 2021.
[38] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Service placement and request routing in MEC networks with storage, computation, and communication constraints," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1047–1060, Jun. 2020.
[39] H. Huang, P. Li, S. Guo, W. Liang, and K. Wang, "Near-optimal deployment of service chains by exploiting correlations between network functions," *IEEE Trans. Cloud Comput.*, vol. 8, no. 2, pp. 585–596, Second Quarter 2020.
[40] Y. Yu, J. Yang, C. Guo, H. Zheng, and J. He, "Joint optimization of service request routing and instance placement in the microservice system," *J. Netw. Comput. Appl.*, vol. 147, 2019, Art. no. 102441.
[41] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Apr. 2021.
[42] R. Borralho, A. Mohamed, A. U. Quddus, P. Vieira, and R. Tafazolli, "A survey on coverage enhancement in cellular networks: Challenges and solutions for future deployments," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1302–1341, Second Quarter, 2021.
[43] I. Baev, R. Rajaraman, and C. Swamy, "Approximation algorithms for data placement problems," *SIAM J. Comput.*, vol. 38, no. 4, pp. 1411–1429, 2008.



Kai Peng received the BE, MS, and PhD degrees from the Huazhong University of Science and Technology, in 1999, 2002, and 2006, respectively. He is currently a professor with the School of Electronic Information and Communications, Huazhong University of Science and Technology, China. His research interests include cloud computing and computer networks.

Liangyuan Wang is currently working toward the EngD degree with the School of Electronic Information and Communications, Huazhong University of Science and Technology. His research interests include edge computing and distributed systems.

Jintao He is currently working toward the ME degree with the School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan, China.



Chao Cai received the PhD degree from the School of Electronic Information and Engineering, Huazhong University of Science and Technology, China. He is an associate professor with the College of Life Science and Technology, Huazhong University of Science and Technology. He has worded as a postdoc with the School of Computer Science and Engineering, Nanyang Technology University, Singapore. His current research interests include mobile computing and sensing, wireless networks, embedded system, digital signal processing, and deep learning.

Menglan Hu received the BE degree in electronic and information engineering from the Huazhong University of Science and Technology, China, in 2007, and the PhD degree in electrical and computer engineering from the National University of Singapore, Singapore, in 2012. He is currently an associate professor with the School of Electronic Information and Communications, Huazhong University of Science and Technology, China. His research interests include cloud computing, computer networks, distributed systems, and mobile computing.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on August 30,2026 at 10:39:07 UTC from IEEE Xplore. Restrictions apply.